Tech Stack

1) Hardware Layer (Core Embedded System)

- **Microcontroller Unit (MCU): Raspberry Pi Pico (RP2040)**
  - Dual-core ARM Cortex-M0+ processor operating at 133 MHz for real-time tactile signal processing.
  - 26 GPIO pins used to interface TTP223 capacitive touch sensors via digital input.

- **Touch Sensing Modules: TTP223 Capacitive Touch Sensors**
  - Detects user touch corresponding to Braille characters (A–Z, 0–9).
  - Provides digital HIGH output to Pico's GPIO, ensuring reliable tactile detection with low latency.

- **Expansion Interface (Optional): PCF8574 I²C Board**
  - Extends GPIO capability for additional sensors or multi-language Braille sets.
  - Enables modular scalability for advanced versions of the device.

- **Power and Connectivity Components**
  - 3.7V–5V rechargeable Li-ion battery with battery indicator and Type-C charging circuit.
  - UART communication pins (TX/RX) for serial data transfer between the microcontroller and host system.

- **Supporting Components**
  - Jumper wires and breadboard circuits for modular testing and assembly.
  - External audio output via host laptop or speaker system for real-time feedback.

2) Firmware and Embedded Software Layer

- **Programming Language: MicroPython**
  - Used for GPIO configuration, interrupt handling, and touch signal acquisition.
  - Implements debounce algorithms and real-time UART communication.

- **Core Modules Used**
  - machine (for GPIO and UART control)
  - utime (for timing operations and debounce logic)

- **Functional Workflow**
  - Touch input → GPIO interrupt → Debounce filter → UART transmission → Host audio response.

- Ensures sub-200 ms tactile-to-audio latency for instantaneous learning feedback.

- **Role**
  - Acts as the primary interface between hardware touch sensors and host software, enabling tactile–auditory synchronization.

3) Software Layer (Host System Application)

- **Language: Python (v3.x)**
  - language for serial communication and audio playback.

- **Libraries and Frameworks:**
  - **PySerial** → Handles UART serial communication from Raspberry Pi Pico to host computer.
  - **PyDub** → Processes and plays corresponding audio files (.mp3 / .wav) for each Braille character.
  - **time / os** → Used for event logging and file management.

- **Audio Feedback Engine:**
  - Receives character data via serial port and plays corresponding phonetic sound file.
  - Provides real-time feedback loop for tactile learning and memory reinforcement.

- **Role:**
  - Converts microcontroller data into auditory output using mapped sound files, simulating Braille-to-speech translation.

4) Communication Protocols and Interfacing

- **UART (Universal Asynchronous Receiver–Transmitter)**
  - Enables serial data transfer between Raspberry Pi Pico and host laptop at 9600 baud rate.
  - Ensures reliable, low-latency character transmission for real-time processing.

- **GPIO (General Purpose Input/Output)**
  - Each GPIO pin is mapped to a unique character sensor for A–Z recognition.
  - Configured in input-pullup mode to avoid floating signals and ensure accurate detection.

- **Future Integration:**
  - Expansion to **I²C** or **SPI** protocols for additional sensors or peripheral modules (e.g., displays or wireless controllers).

5) Audio and Learning Feedback System

- **Audio Files:**
  - Pre-recorded .mp3 files mapped to English alphabets (A–Z) and numerals (0–9).
  - Stored locally on host system for fast access and playback.

- **Audio Processing Logic:**
  - Data → Character Recognition → File Fetch → Play Output.
  - Delivers synchronized tactile–audio feedback for efficient cognitive learning.

- **Enhancements (Planned):**
  - Dynamic voice modulation and multilingual support for regional Braille learning.
  - Integration with voice-based AI assistant for guided learning experience.

6) Power and System Management

- **Battery System:**
  - Rechargeable Li-ion battery providing stable DC power to all components.
  - Battery indicator monitors charge level to prevent over-discharge.

- **Charging Interface:**
  - Type-C USB charging module for quick recharge and durability.
  - Enables safe current regulation for longer battery life.

- **Power Optimization:**
  - Sleep-mode management for touch sensors when idle.
  - Low-power consumption microcontroller design for extended usage sessions.

7) Development Environment

- **IDE:** Thonny IDE / Mu Editor (for MicroPython firmware development).

- **Programming Interface:** Visual Studio Code (for Python host application).

- **Version Control:** Git + GitHub (for code maintenance and collaborative versioning).

- T**esting Tools:** Serial Monitor & UART Debugger (for data validation and signal integrity testing).

- **Audio Editing Software:** Audacity (for generation and normalization of phonetic audio files).

8) Future Software & Hardware Integration

- **Mobile Application Framework:**
  - Flutter / Kivy for Android integration, enabling audio customization and progress tracking.

- **Wireless Connectivity:**
  - ESP32 integration for Bluetooth-based data exchange and wireless learning interface.

- **Cloud Support:**
  - Firebase / MQTT for storing user progress data and delivering content updates remotely.

- **AI Integration (Future Enhancement):**
  - Adaptive audio feedback using speech recognition and machine learning for personalized learning modules.